ADARE: Adaptive Multi-Objective Scheduling for Edge/Fog/Cloud

1st Roland Cédric TAYO
CY Cergy Paris University, France
rolandcedric.tayo@yahoo.fr

2nd Sonia YASSA
CY Cergy Paris University, France
sonia.yassa@cyu.fr

***Abstract*—Yesterday we were in the era of man interacting with the machine. Today, the machine interacts with other machines. The purpose of these exchanges is to produce and provide data even faster than yesterday. In order to achieve this feat, the machines distribute the tasks among themselves. Several algorithms make this possible, such as NSGA-III, which turns out to be one of the most effective. But we notice that most of these algorithms remain static in their approach.**

This paper presents ADARE (Adaptive Deep-reinforced Auto-learning Resource Allocator Engine), a dynamic approach of adjusting its own parameters during a processing, in order to propose better distributions.

ADARE and NSGA-III taken on the basis of identical parameters (within 6 runs) have shown that ADARE wins the competition by proposing scores going up to $18/18$ in terms of quality and $15/15$ in terms of core-quality, for the datasets "Montage_25", "CyberShake_30" and "Epigenomics_24". That is gains going up to $+36.18\%$ (core quality), $+9.45\%$ (best-objective metrics), and $+9.17\%$ (runtime)

We will explore throughout this document the 3 major algorithmic contributions of ADARE as well as the results of the tests conducted for our study.

***Index Terms*—Edge computing, Fog computing, Cloud computing, Multi-objective optimization, NSGA-III, Adaptive evolutionary algorithm**

I. Introduction

A. Background and Motivation

The Internet of Things (IoT) has invaded and conquered the routine of man. Behind these objects are hidden computing units represented by Edge, Fog and Cloud infrastructures. The selection of the ideal infrastructure to perform a computation influences the computation time, the response time, the energy consumption and by ricochet the financial costs.

These perpetually conflicting objectives are the constant optimization challenge that multi-objective optimization algorithms must overcome [1], [2].

B. Goals

The fundamental goal is to find a common ground between these objectives, to optimize them. ADARE explores a dynamic and adaptive approach, while remaining fair-play during the evaluation. The goals are:

- adapt operator behavior during search,
- keep Pareto diversity while improving convergence,
- improve objective extremes (especially makespan and latency),
- avoid runtime degradation.

C. Paper Structure

We have structured the paper as follows: Section II summarizes related approaches. Section III defines the optimization model. Section IV presents ADARE and its main algorithms. Section V gives the experimental setup. Section VI reports results. Section VII concludes.

II. Related Work

A. Heuristic and Deterministic Schedulers

Classical schedulers such as certain list-based heuristics are often very efficient when it comes to producing fast results. In heterogeneous edge/fog/cloud environments, this kind of algorithm would imply poor energy-cost trade-offs. Deterministic rules seriously lack efficiency when facing the highly variable characteristics of workloads.

B. Evolutionary Many-Objective Methods

Evolutionary methods balance conflicting objectives by maintaining non-dominated populations. NSGA-II and NSGA-III are standard baselines. NSGA-III turns out to be particularly suited to many-objective situations via reference-point selection [2]. But, when it comes to workflow scheduling, static crossover/mutation parameters can drastically reduce efficiency. Key operators could degrade the quality of solutions in later generations.

C. Learning-Guided Hybrids

Recent hybrid methods try to combine search and learning signals. But this approach often implies external pre-training or overly complicated and costly hyperparameter tuning. ADARE keeps the main learning loop light and online. Instead of replacing the evolutionary core, it injects adaptive mechanisms inside the loop itself : a crossover that adapts to the situation, a density-sensitive mutation (nothing is done randomly), and an archive-driven steering with final candidates converging toward the objectives. We learn better without building a factory of gas.

III. Problem Modeling

A. System and Decision Model

We schedule a workflow DAG $G = (V, E)$ over heterogeneous resources R (edge/fog/cloud). Each task $v_i \in V$ has computational load and data transfer requirements. Each resource $r_k \in R$ has different processing rate, communication bandwidth, cost, and power profile.

An individual x is a task-to-node assignment:

$$x = (x_1, \ldots, x_{|V|}), \quad x_i \in \{1, \ldots, |R|\}. \tag{1}$$

B. Objective Functions

The objective vector is:

$$\min \mathbf{f}(x) = \big(f_{\text{makespan}}, f_{\text{latency}}, f_{\text{cost}}, f_{\text{energy}}\big). \tag{2}$$

Main components are:

- $f_{\text{makespan}} = \max_i FT(v_i)$,
- $f_{\text{latency}} = \sum_i \big(FT(v_i) - RT(v_i)\big)$,
- $f_{\text{cost}} = \sum_i ET(v_i, x_i) \cdot C(x_i)$,
- $f_{\text{energy}} = \sum_i ET(v_i, x_i) \cdot P(x_i)$.

Here FT is finish time, RT is ready time under precedence constraints, and ET is execution time on the assigned node.

C. Optimization Target

Our objectives are to :

- a high-quality non-dominated front (HV, IGD, spacing, epsilon, coverage),
- robust objective extremes (best makespan, latency, cost, energy),
- competitive runtime.

IV. ADARE Methodology

A. Simple Structure Design

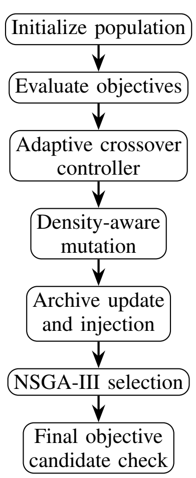


Fig. 1: Simple ADARE structure.

B. Contribution 1: Adaptive Crossover Control

This controller will progressively elect the best rewarded operators along the previous generations. The UCB term prevents the system from locking too early on a single operator.

C. Contribution 2: Density-Aware Mutation

The more diversity decreases, the more mutation is stimulated in order to avoid any premature convergence. The structures obtained from previous generations are thus preserved.

Algorithm 1: Adaptive Crossover Selection (RL-guided)

Input: Parents p_1, p_2, operator values Q, usage counts N, generation g
Output: Children c_1, c_2, updated Q, N

$\epsilon \leftarrow$ linear_schedule(g);
if *rand()* $< \epsilon$ **then**
  $k \leftarrow$ random operator;
else
  $k \leftarrow \arg\max_i \left(Q_i + \sqrt{\frac{2\ln(\sum_j N_j + 1)}{N_i + 1}}\right)$;
Apply crossover operator k on (p_1, p_2) to get (c_1, c_2);
$r \leftarrow \phi(\text{best}(p_1, p_2)) - \phi(\text{best}(c_1, c_2))$;
$Q_k \leftarrow (1-\alpha)Q_k + \alpha r$;
$N_k \leftarrow N_k + 1$;

Algorithm 2: Density-Aware Adaptive Mutation

Input: Individual x, diversity d, generation g, task-node rankings
Output: Mutated individual x'

$B \leftarrow$ dynamic mutation budget based on (g, d);
for $b = 1$ **to** B **do**
  pick task index t;
  if *rand()* $< p_{heur}$ **then**
    assign x_t from top-ranked nodes for task t;
  else
    assign x_t to a random node;
apply low-probability gene-level mutation pass;
$x' \leftarrow x$;

D. Contribution 3: Archive and Final Objective Candidates

The most important point is to separate roles: Pareto quality is measured on the stable final front P, while objective extremes are validated on P^{obj}. This keeps front quality intact while allowing targeted objective improvements.

V. Experimental Setup

The comparison protocol is strictly the same for ADARE and NSGA-III to avoid any bias.
Benchmarks: Montage_25, CyberShake_30, Epigenomics_24.
Metrics: HV, IGD, spacing, epsilon, coverage, runtime, and best objective values.

A. Fairness Controls

For each run, we make sure that both algorithms start with the same initial population. For that, the same seed is used at the start. The evaluator, the order of objectives, the hardware context and the generation budget are all identical. We thus protect ourselves from any hidden advantage, linked to initialization or to differences in the way metrics are implemented.

B. Statistical Protocol

We retrieve, for each benchmark, means and standard deviations over 6 runs. We also perform paired Wilcoxon tests to observe whether gain values are statistically significant across shared-seed runs.

Algorithm 3: Archive-Guided Final Objective Pool

Input: Final population P, archive A, candidate keys K
Output: P (quality front), P^{obj} (objective-extreme pool)

$P^{obj} \leftarrow P \cup A$;
foreach $k \in K$ **do**
    evaluate candidate k;
    if k *improves best makespan or best latency or best energy* **then**
        add k to P^{obj};
return P for Pareto-quality metrics and P^{obj} for objective-best metrics;

TABLE I: Shared Configuration

Parameter	ADARE	NSGA-III
Population size	100	100
Generations	70	70
Crossover probability	0.85	0.85
Mutation probability	0.20	0.20
Gene mutation probability	0.08	0.08
Runs per benchmark	6	6

VI. Results and Discussion

TABLE II: Global Results (Latest 6-Run Campaign)

Benchmark	Core Gain	Obj. Gain	Time Gain
Montage_25	+36.31%	+14.78%	+6.93%
CyberShake_30	+20.48%	+10.59%	+12.38%
Epigenomics_24	+51.75%	+2.98%	+8.18%
Mean	**+36.18%**	**+9.45%**	**+9.17%**

Global outcomes: ADARE wins 18/18 quality slots and 15/15 core-quality slots. It also improves previously weak extremes:

- CyberShake_30: makespan_best +5.34%, latency_best +8.90%.
- Epigenomics_24: makespan_best +1.33%, latency_best +4.99%.

Pareto and convergence plots agree : ADARE covers more interesting and exploitable trade-off zones and reaches higher extreme values. All of this while maintaining its speed across all three benchmarks.

A. Metric-by-Metric Interpretation

The global improvements are not caused by a single indicator. HV, IGD, spacing and epsilon all progress across the three workflows. This is an excellent demonstration of better convergence and a good maintenance of diversity. Coverage gains are exceptionally significant when it comes to Epigenomics_24 and Montage_25. ADARE outperforms a larger portion of the solutions generated by NSGA-III in these cases.

Runtime remains positive across the three workflows, despite the adaptive controls and the archive logic. This is mainly explained by the dynamic operator selection and a controlled mutation budget, which avoid costly and unnecessary transformations in the late generations.

The most significant gains on extreme objective values appear on cost and latency in CyberShake_30. Regarding Montage_25, we have a strong improvement of the Makespan. Gains are more modest for Epigenomics_24 on energy_best, but they remain positive and stable regardless of the run.

B. Significance Overview

These 6 paired runs already allow us to observe significant differences on several metrics, but not on all of them. This is a normal behavior in multi-objective scheduling : some metrics (such as spacing or epsilon) can vary more from one run to another depending on the shape of the Pareto front. Increasing the number of runs and testing other workflow sizes would certainly help stabilize the statistical conclusions even further.

C. Practical Takeaways

From an engineering standpoint, ADARE can easily replace NSGA-III. This could be particularly interesting in cases where the scheduler must produce both a good front and reliable extreme candidates. The final candidate per objective mechanism is particularly useful in production-close contexts, with operators being able to focus on a critical metric (a deadline or a response time) while keeping alternative trade-offs accessible.

D. Reproducibility Notes

Main command used for the test (instead of a "make" command):

- python main.py --benchmarks Montage_25 CyberShake_30 Epigenomics_24 --runs 6

Generated reports:

- output/summary/global_comparison_report.txt
- output/<workflow>/reports/*_comparison_report.txt

Generated figures used in this paper:

- Figures/pareto_test_montage25.png
- Figures/pareto_test_cybershake30.png
- Figures/pareto_test_epigenomics24.png
- Figures/convergence_test_montage25.png
- Figures/convergence_test_cybershake30.png
- Figures/convergence_test_epigenomics24.png

E. Threats to Validity

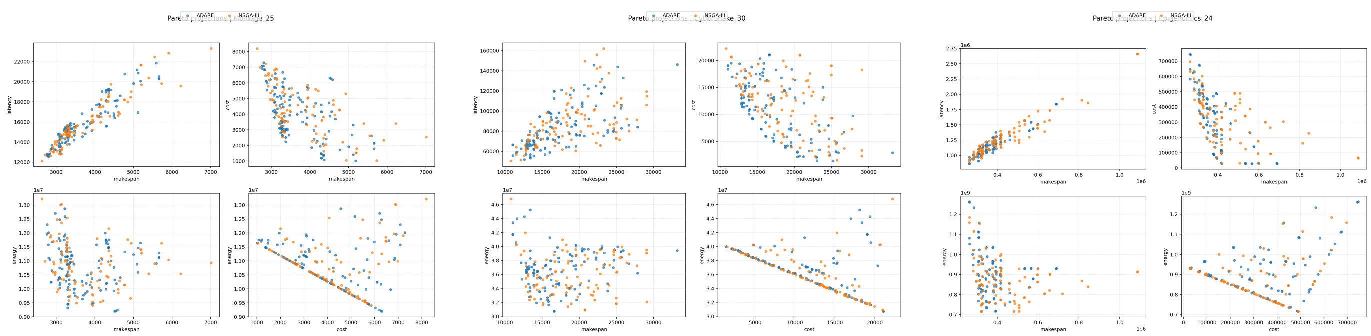
Tests are conducted on three representative workflow families, but it would be worth testing more sizes and different infrastructure profiles. Also, statistically significant data comes with a larger number of runs. 6 runs constitute a good base, but this is not a definitive ceiling for drawing solid conclusions.

VII. Conclusion

The comparative study we conducted demonstrates that ADARE improves over NSGA-III through three concrete algorithmic contributions : an adaptive crossover control, a density-sensitive mutation, and an archive-guided final selection mechanism. With a fair protocol, ADARE remains

TABLE III: Detailed Metrics Snapshot (mean±std over 6 runs)

Benchmark	Metric	ADARE	NSGA-III	Gain (%)
Montage_25	HV	0.91163 ± 0.0108	0.864108 ± 0.0224	+5.50
Montage_25	IGD	0.0621207 ± 0.00555	0.0644155 ± 0.00447	+3.56
Montage_25	Time	1.15959 ± 0.034	1.246 ± 0.069	+6.93
Montage_25	Makespan_best	2282.78 ± 0	2876.98 ± 147	+20.65
Montage_25	Latency_best	11160 ± 0	12744.5 ± 535	+12.43
CyberShake_30	HV	0.929627 ± 0.00732	0.89443 ± 0.0104	+3.94
CyberShake_30	IGD	0.0698127 ± 0.00494	0.0745182 ± 0.00348	+6.31
CyberShake_30	Time	1.29986 ± 0.0803	1.48357 ± 0.123	+12.38
CyberShake_30	Makespan_best	10806.6 ± 0	11416.4 ± 515	+5.34
CyberShake_30	Latency_best	47940 ± 0	$52626.2 \pm 1.72 \times 10^3$	+8.90
Epigenomics_24	HV	1.1125 ± 0.00156	1.1046 ± 0.00345	+0.72
Epigenomics_24	IGD	0.0381703 ± 0.00329	0.0401564 ± 0.00321	+4.95
Epigenomics_24	Time	1.18571 ± 0.0242	1.29134 ± 0.069	+8.18
Epigenomics_24	Makespan_best	261644 ± 0	$265167 \pm 7.4 \times 10^3$	+1.33
Epigenomics_24	Latency_best	830721 ± 0	$874313 \pm 3.5 \times 10^4$	+4.99



(a) Montage_25 (b) CyberShake_30 (c) Epigenomics_24

Fig. 2: Pareto projections regenerated from the latest 6-run test campaign.

TABLE IV: Quality Metric Gains (%) of ADARE over NSGA-III

Metric	Montage	CyberShake	Epigenomics
HV	+5.50	+3.94	+0.72
IGD	+3.56	+6.31	+4.95
Spacing	+16.73	+1.89	+27.83
Epsilon	+21.35	+17.84	+5.64
Coverage	+134.40	+72.42	+219.59
Runtime	+6.93	+12.38	+8.18

TABLE V: Best-Objective Gains (%) of ADARE

Metric	Montage	CyberShake	Epigenomics
Makespan_best	+20.65	+5.34	+1.33
Latency_best	+12.43	+8.90	+4.99
Cost_best	+23.27	+26.08	+5.48
Energy_best	+2.76	+2.03	+0.14

TABLE VI: Significant Wins Summary (Wilcoxon, $p < 0.05$)

Benchmark	All Quality	Core Quality
Montage_25	4/6	3/5
CyberShake_30	3/6	2/5
Epigenomics_24	4/6	3/5
Total	**11/18**	**8/15**

superior in quality and runtime across all three workflow families. We believe that an adaptive model is the answer to the extremely heterogeneous nature of environments.

REFERENCES

[1] K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms*. Wiley, 2014.

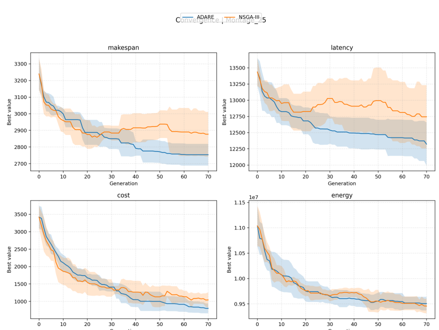
[2] K. Deb and H. Jain, “An Evolutionary Many-Objective Optimization Algorithm Using Reference-Point-Based Nondominated Sorting Approach, Part I,” *IEEE Trans. Evol. Comput.*, vol. 18, no. 4, pp. 577–601, 2014.

[3] J. Durillo et al., “Multi-objective workflow scheduling in cloud systems,” *Future Generation Computer Systems*, 2014.

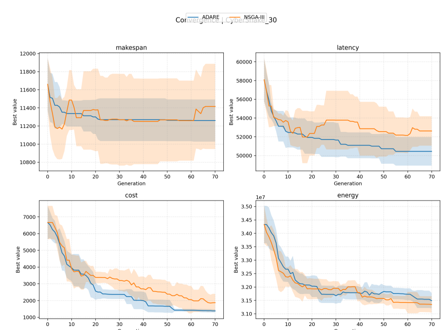
[4] X. Xu et al., “Many-objective scheduling with NSGA-III variants,” *Applied Soft Computing*, 2018.

[5] H. Wu et al., “A Survey of Task Scheduling in Cloud and Fog Computing,” *Future Generation Computer Systems*, 2019.

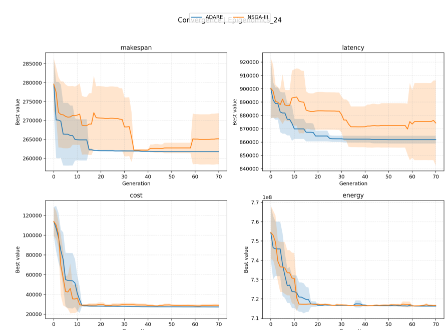
[6] M. Goudarzi et al., “Resource Management in Fog/Edge Computing: A Survey,” *ACM Computing Surveys*, 2021.



(a) Montage_25



(b) CyberShake_30



(c) Epigenomics_24

Fig. 3: Convergence curves regenerated from the latest 6-run test campaign.